```
# RiffMates/RiffMates/settings.py
...
DATABASES = {
    'default': {
        'ENGINE': 'django.db.backends.sqlite3',
        'NAME': BASE_DIR / 'db.sqlite3',
    }
}
```

Django is capable of interacting with multiple databases at one time. Doing so is outside the scope of this book, but this is why there is a nested dictionary inside the `DATABASES` configuration. You can end up with more than just `default` configured.

To use a different database, you change the `ENGINE` subproperty to point at a different backend. The other key–value pairs in the configuration are database specific. SQLite, being a file-based database, has a simple configuration: just the filename. The other databases interact over the network. To configure for a PostgeSQL database, you provide an `OPTIONS` dictionary instead of the `NAME` argument. The contents of that dictionary specify a service configuration file and a password storage file. The service configuration file contains network information, a username, and the database name to connect with. The password storage file contains your connection password. More details on interacting with specific databases can be found in the Django documentation: https://docs.djangoproject.com/en/dev/ref/databases/.

B.7 Caching

For a small project with a few users, your web server will likely be snappy enough that you'll get sufficient performance for your page. As you scale, you may need to introduce caching. A *cache* records the previous output of a computation and serves the recording up instead of rerunning the computation.

In Django's case, the computation is the view. With caching turned on, the first time a user hits a view, the result gets stored in the cache. Subsequent visits to the same view get served out of the cache instead, reducing the amount of code that needs to be run.

Django can use three different types of caches, based on

- Servers
- Databases
- Files

A cache server is a dedicated piece of software that provides caching functionality. Two popular choices are Memcached and Redis. Your hosting provider may offer these as add-on packages or cloud-based instances.

You can also have Django use your database to cache content. To do this, you need to create an extra table in your database to store the content; this can be accomplished using the `createcachetable` management command.

Alternatively, you can use files to store the cached contents. In this case, you point Django at a directory to use for the cache files. This directory should not be inside your project.